# CherryRiver SAQ Pipeline — AI-First Engineering Video Notes

> Use this file as your reference during the Loom recording. Talk through each section in order. Each block = ~2 minutes on screen.

---

## SECTION 1 — The Problem & Why It Was Difficult (3 min)

### Business Context

- Client: Cherry River, a wine/spirits distributor in Quebec
- The SAQ (Société des alcools du Québec) is the government-controlled liquor board
- Every week, Cherry River needs sales data, store inventory, and warehouse inventory for their 76 products
- Manual process: someone logging into SAQ B2B portal, clicking through 6 reports, downloading, processing in Excel, uploading somewhere

### The Challenge I Took On

**Build a fully automated weekly pipeline that:**

1. Logs into the SAQ B2B portal via Selenium
2. Downloads 6 CSV/ZIP reports
3. Processes and filters them to Cherry River's 76 product codes
4. Uploads the data to Supabase
5. Powers a Retool dashboard with demand prediction

### Why This Was Non-Trivial

- The SAQ portal is a legacy Java/JSP system — no public API
- Selenium on a GitHub Actions Linux runner is unreliable by default
- The CSV files use semicolon delimiters and latin-1 encoding (not standard)
- Different tables need different deduplication strategies (sales ≠ inventory ≠ orders)
- The prediction algorithm needed anomaly removal to be useful
- I had never done Selenium in a cloud CI/CD environment before

---

## SECTION 2 — How I Used AI To Help Solve It (5–6 min)

### 2A. Architecture Design with Claude

Before writing a line of code, I described the business problem to Claude and asked it to help me design the data flow. Claude laid out the 3-step pipeline concept:

```
Scraper → Unzip → Upload
```

And helped me decide which Supabase tables needed APPEND vs UPSERT vs deduplication logic — which became the foundation of the whole system.

**Show:** `run_weekly_pipeline.py` — the orchestrator Claude helped design

---

## 2B. The Hardest Bug — Headless Chrome Failing on GitHub Actions

**The symptom:** The pipeline runs perfectly on my Windows machine. I push to GitHub, the Monday cron fires, and the job times out after 30 minutes with no downloads.

**My first instinct:** Increase timeouts, add more sleeps. This didn't help.

**I asked Claude:** "My Selenium scraper works locally but fails silently on GitHub Actions Linux runner. The job doesn't error — it just hangs. What are the most common causes?"

Claude pointed me to three things I hadn't considered:

1. Chrome in headless mode doesn't fully initialize the rendering engine, so JavaScript-heavy pages can hang
2. `--disable-dev-shm-usage` is critical on Linux (shared memory too small by default)
3. Xvfb (X Virtual Framebuffer) — running Chrome in "visible" mode against a virtual display — is more stable than headless for complex sites

**The fix — commit `fc0c024`:**

```
# Before:
headless = True  # CI mode

# After:
headless = False  # Use visible Chrome with Xvfb virtual display
```

```
# Workflow addition:
- name: Start Xvfb
  run: |
    Xvfb :99 -screen 0 1920x1080x24 > /dev/null 2>&1 &
    sleep 3
  env:
    DISPLAY: ':99'
```

`.github/workflows/saq_weekly_update.yml` lines ~65–75, then `saq_data_scraper.py` line 544

**The insight AI gave me that I hadn't considered:** The problem wasn't timeout values — it was that headless Chrome on Linux legitimately behaves differently for JavaScript-heavy sites. Switching to Xvfb visible mode is a known production pattern for scraping legacy enterprise portals.

---

## 2C. The Chrome Driver 3-Tier Fallback

**Problem:** On my Windows machine I have `chromedriver.exe` locally. On GitHub Actions Linux, there's no chromedriver in PATH by default. On other devs' machines, it might be installed differently.

**I asked Claude:** "How can I make the Chrome driver initialization work across Windows dev, Linux CI, and any machine without manual setup?"

Claude helped me write a 3-tier fallback:

```
# Method 1: Local chromedriver.exe (Windows dev)
# Method 2: System PATH (Linux with package install)
# Method 3: Selenium 4.6+ auto-download (zero setup)
```

**Show:** `saq_data_scraper.py` lines 95–153

---

## 2D. The CSV Format Surprises

When I first ran the code, it crashed with a UnicodeDecodeError and the data looked wrong — comma-separated when there were no commas.

**I asked Claude:** "Why is my CSV reader not parsing SAQ files correctly? The data looks like one long string per row."

Claude immediately diagnosed two issues:

1. SAQ uses `;` (semicolon) — European standard, not `,`
2. SAQ exports are `latin-1` encoded, not UTF-8 — common with legacy French government systems

```
# The fix Claude identified:
with open(filepath, 'r', encoding='latin-1') as f:
    reader = csv.DictReader(f, delimiter=';')
```

`scripts/saq_weekly_update.py` lines 98–105

---

## 2E. Deduplication Strategy — Different for Each Table

**Problem:** The pipeline runs every Monday. What happens if it runs twice in one week? What happens when I re-run a past week's file?

**I asked Claude:** "I have 4 different tables with different business semantics. Help me think through the right insert strategy for each."

Claude walked me through this:

- **Sales (ventes):** `APPEND only` — a week either exists or doesn't. Check `(annee, periode, semaine)` once, skip if exists
- **Store inventory:** `UPSERT` — each week replaces last week's snapshot for that store+product combo

- **Warehouse inventory:** `UPSERT` — same logic, different key fields
- **Orders (commandes):** `SELECT before INSERT` per order number — orders have unique IDs, never re-insert same order

`scripts/saq_weekly_update.py` — `process_ventes()` skip logic (~line 115), then upsert patterns (~lines 181–187)

---

### 2F. The Demand Prediction View — Patrick's Anomaly Algorithm

The client (Patrick) wanted a view that says: "How many weeks of stock do we have, and should we order now?"

The naive approach is `stock / average_weekly_sales`. The problem: SAQ runs promotions that spike sales 3–5× for one week. That spike makes the average look inflated, which makes weeks-of-stock look lower than reality → false "COMMANDER" alerts.

**I explained the business problem to Claude** and asked it to help write the SQL for a cleaned average:

Claude helped me implement a 2-pass anomaly removal:

```
-- Pass 1: Calculate a "rough baseline" by excluding the top 4 weeks per product
-- Pass 2: Remove ALL weeks where sales > baseline × 1.5
-- Pass 3: Average the remaining "normal" weeks
```

**Special edge case Claude caught:** If a product has fewer than 5 weeks of data, there's not enough to calculate a baseline — keep all weeks.

`scripts/create_dashboard_views.sql` lines 22–113 (v_po_prediction)

---

### 2G. The Large SQL File Committed to Git

At one point I committed a large database dump file (~50MB) to the repository by mistake. GitHub rejected the push.

**I asked Claude:** "I accidentally committed a large SQL file and git push is failing. How do I remove it from git history without losing my other changes?"

Claude walked me through `git filter-branch` vs `git filter-repo`, guided me to the safer option, and helped me update `.gitignore` to prevent this from happening again.

**Commit:** `be0f03f` — "Remove large SQL file and update .gitignore"

`.gitignore` — the `*.sql` and `SAQ Documents*/` entries

---

## SECTION 3 — What I Validated Manually (2 min)

### End-to-End Test on Windows

1. Ran the full pipeline locally with real SAQ credentials
2. Watched Chrome open, navigate, download all 6 files
3. Verified CSVs in `SAQ Documents 2/` — correct row counts, correct product codes only
4. Checked Supabase tables — rows appeared, no duplicates
5. Ran pipeline again same day — verified sales table was NOT re-inserted (dedup working)

### GitHub Actions Validation

1. Merged the Xvfb fix to main
2. Manually triggered the workflow (`workflow_dispatch`)
3. Watched the run logs in GitHub Actions UI — Chrome started, Xvfb initialized, downloads completed
4. Checked the uploaded artifacts (all 6 files present)
5. Verified Supabase — new rows for the correct week

### Prediction View Validation

1. Ran v_po_prediction query manually in Supabase SQL editor
2. Cross-checked one product's "cleaned average" by hand — pulled raw weekly data, removed spikes, calculated manually, compared to view output
3. Confirmed edge case: product with 3 weeks of data gets full average (no anomaly removal)

### One AI Suggestion I Rejected

Claude initially suggested using `git filter-repo` for the large file removal (cleaner, but requires installing a non-standard tool). I rejected this because the GitHub Actions runner doesn't have it, and I wanted a solution I could run anywhere. I used `git rm --cached` + `git rebase` instead — simpler, no extra dependencies.

---

## SECTION 4 — Final Outcome: Before vs After (2 min)

| | **Before** | **After** |
|---|---|---|
| **Process** | Manual login, 6 manual downloads, Excel processing | Fully automated, runs every Monday 3 AM EST |
| **Time per week** | ~45 min manual effort | 0 min — pipeline runs unattended |
| **Error rate** | High (manual copy-paste errors) | Near-zero (deduplication logic prevents double-inserts) |
| **Data freshness** | Whenever someone remembered to run it | Every Monday before business starts |
| **Demand prediction** | None | v_po_prediction view with anomaly-cleaned weekly avg |
| **CI/CD** | N/A | GitHub Actions, free tier, runs on cron |
| **Reliability** | N/A | Xvfb fix eliminated all timeout failures |

## SECTION 5 — What I'd Do Differently (1 min)

1. **Use Playwright instead of Selenium.** Playwright has better async support, more reliable headless mode, and first-class Python API. I'd use it for any new scraping project.

2. **Add structured logging.** Right now the logs are `print()` statements. For a production system I'd use Python's `logging` module with log levels and timestamps to make GitHub Actions logs easier to search.